O&G Workbench v0.8.37 — Independent Suite Audit

Read-only audit · 27 October 2026 · Authoritative baseline: uploaded v0.8.37 ZIP

Executive assessment

The suite has substantial useful shared infrastructure and meaningful tests. It does not justify a wholesale UI or renderer rewrite. The release does have reproducible data-loss, numerical-unit, cross-module refresh, asset-persistence and export-scaling defects that should be addressed before v1.0.

The most urgent findings are geosteering output collisions across wells (C01), non-atomic replacement of geosteering crossing tops (C02), and destructive zone-result publication when batch jobs share a source (H04). Unit interpretation also needs an explicit contract: fraction scale (H01), log-versus-well depth units (H02), and production denominator units (H03). These are separate problems; changing a geological equation is not the proposed remedy.

No application files, database schema, algorithms, UI or supplied project data were modified. All 524 files from the archive remained byte-identical after testing, with no extra files in the extracted source tree. Synthetic projects, probes, environments, logs and reports were kept outside that tree. The source archive SHA-256 is c1a7836fd09390f7170e7740cf4e52b886e472b68942704eab9b7d6e91222744.

How to read the evidence

- **Confirmed defect / reproduced:** the current source supports the finding and an isolated probe exercised the failure, sometimes by injecting a failure at a specific publication boundary. Injection establishes the behavior if that operation fails; it is not evidence that a client has already suffered it.

- **Confirmed policy / scale risk:** the allocation or behavior exists in the code, but actual impact depends on project size and use. Arithmetic memory estimates are not measured application RSS.

- **Architectural concern / Cleanup:** a maintenance or consistency issue, not proof of incorrect geology. No personal naming/formatting preference is classified as a defect.

- Confidence applies to the stated evidence. No supplied client database, 38,000-file raster library, production-scale SEG-Y volume, or historical client saved state was available. No claim of exhaustive correctness, complete line-by-line inspection, Windows certification or security certification is made.

This was a suite-wide audit of entry points, data models, authoritative read/write paths, cache dependencies, representative renderers and numerical routines, imports/exports, settings and the test suite. The inventory contains 371 Python files / 170,041 lines, including approximately 43,327 test lines. A suite-wide audit of this size is not a proof of every reachable branch. Recommendations below identify bounded changes; none were implemented.

Verification performed

Check	Outcome
Full existing suite, Linux/Python 3.12/Tcl-Tk 9.0.4 with a real virtual X display	1,424 passed; 9 failed; 8 skipped; 285 passing subtests; 776.25 seconds
Targeted rerun of those nine failures with a writable raster-cache override	4 passed; 5 failed; 9.97 seconds
Isolated data/asset probes	Confirmed filename collision, partial geosteering publication, zone-result deletion, future-schema downgrade, mixed-unit behavior, stale stores, truncated export, incomplete fingerprint, grid partial save, external-file deletion, malformed grid discovery, posting cache and interrupted shapefile overwrite
Performance probes	Workbook append scaling; posted-value artist construction/draw scaling; timings are indicative on this machine, not client-scale service targets
Archive/source integrity	524 original files unchanged; no added source-tree files
Windows executable/installer	Not run on this Linux host; Windows CI/build scripts inspected

The initial full run used the runtime raster cache default, which is outside the writable sandbox. A supported OGW_RASTER_CACHE=/tmp/audit0837-raster override resolved three raster failures on rerun. A spreadsheet deferred-fill test also passed on rerun and is timing-sensitive. Those results are reported separately; the full suite was not rerun and is not described as all passing.

Five remaining UI/toolkit failures

Test	Evidence and disposition
Correlation: enlarged slip box preserves identical old pixels	Small exact-pixel differences remain: 28 pixels in the full run, 83 in the isolated rerun. Image invariance needs backend/Windows comparison; it is not dismissed as harmless or promoted to a geological defect without that check.
Form mouse wheel	Reproduced. Synthetic Button-5 arrives as num=9 on this Tk 9 runtime; the code interprets it as upward. See L03.
Cross Section GridDialog must scroll when small	The 193 px body fits a 209 px canvas; all buttons are reachable. Requiring a scrollbar in that case is not an accessibility requirement.
Seismic Auto Pick dialog must scroll when small	The 187 px body fits a 211 px canvas; the test requires scrolling irrespective of fit. Manual Windows accessibility remains to be checked.
Batched Treeview insertion equals individual insertion	The data rows match; Tk 9 exposes empty text as () for one insertion path versus "" for the other. Normalize representations or define the toolkit contract before treating this as data loss.

Segoe UI is absent on this host, so font/layout assertions cannot certify the Windows appearance. Seven skipped tests need unavailable real-log images; one is a Windows Explorer-drop test. Generic raster/digitizer tests did run. The suite has valuable coverage of atomic pay edits, sources, alias/curve precedence, selections, migrations, finite BHL fallback, grid planes, SEG-Y variants, persistence and rendering. Its gaps are identified below rather than inferred from pass counts alone.

Scope and architecture assessment

Area	Paths inspected / exercised	Main result
Geo / Well Manager	database.py, bulk_import.py, mapped_import.py, main_window.py, sheet/depth/API/location workflows	Forward-schema guard missing; shared ID/source/depth infrastructure is useful. Batch Tcl row insertion has a toolkit-representation test failure.
Map	window/view, datastore, changes, culture, grids, sources, drawings, posting, cache/spatial/text paths	Grid persistence/export issues; stale external posting items; value-post artist scaling. Compact store, dependency refresh, spatial culling and batched header text already exist.
Petro	workspace, geo_link, calculations/formulas, plotting, curves, tracks, pay and file I/O	Fraction/depth-unit hazards cross this boundary. Save-before-switch and outside-curve reconciliation already exist; do not replace them speculatively.
Seismic	ogcore/segy, wells, project, app, mapping, sections/cube and interpretation	Fingerprint omits geographic identity. Memory-mapped/chunked trace reading, bounded section cache and close paths are present.
Geosteering	project, job/state, model, wells, app/results and top write paths	Same-name file collision and non-atomic results publication. Type-well/lateral unit conversion exists in job state; that does not solve the shared raw-log/project-axis mismatch.
Correlation	store, model, table, app, session/picking/raster flows	Stale fact/curve caches and aggregate retention; exact-pixel test remains unresolved.

Area	Paths inspected / exercised	Main result
Cross Section	store, window, model/layout/render/log-display/fills/export	Same stale-cache concern, raw depth-unit boundary, per-store memory retention; normal small-dialog fit was confirmed.
Raster / Digitizer	ogcore/raster, digitize, autoreg, ograster calibration, overlays, bulk LIC, digitize UI	Per-image caching and safe image/file contexts exist; aggregate reader retention and partial asset publication remain risks. Some real-image interaction tests are skipped because the images are absent.
Batch	batch_curves, batch_zones, curve_calc, pay, resampling, aliases, conflict policies	Mixed axes and source-scoped result deletion reproduced. Per-well errors/cancellation and run reports exist.
Production	imports, source choice, summaries, unit factors, spans, metric mapping	Per-foot unit defect; ambiguous gas units; cold all-month dictionary materialization. Source-aware duplicate handling and revision-aware reuse already exist.
Infrastructure / persistence	project paths/copy, database transactions, safeio, cache, background, handoff, selection/pay notifications	Good foundations; file/database operations need recoverable logical publication. Advisory locks are not transactional concurrency control.
Import / export	LAS, CSV/XLSX, surveys, XYZ/ZMAP, grids, shapes, SEG-Y and raster controls	Specific defects in ZMAP extent, workbook scaling, shapefile overwrite atomicity and malformed-grid isolation; no blanket claim that all importers are unsafe.
Security / distribution	formula AST evaluator, deserialization, subprocess paths, temporary writes, dependencies and Windows build	No unrestricted project-formula eval/pickle-loading path identified. Destructive asset paths need ownership checks. Build/runtime dependencies are not a reproducible lock; Windows build was not validated here.

Shared concepts: preserve existing services, close the concrete gaps

The project database and ogcore already centralize API matching, normalized interpretation sources, aliases, depth conversion, pay, selections, color bars, project-folder roles, caches and background execution. Keep database well ID/UID as internal identity and treat API as matching/presentation metadata. Existing API rules deliberately distinguish sidetracks/completions and are covered by tests; do not replace them with prefix matching as a cleanup.

Independent view models and renderers are reasonable: Correlation’s paper-log layout, Cross Section geometry, Petro tracks, Map collections and Geosteering trajectories have different needs. The shared gaps are the data contracts and invalidation policies in H01–H04/H09/M04, not the fact that the views have separate classes. A monolithic shared renderer would increase regression risk without resolving those defects.

database.py, Map windows and several UI builders are large. Their size alone is not a finding. There is no recommendation to split them into mixins, rename classes wholesale or merge renderers. Consider extracting only a stable data-only contract when a confirmed fix needs it. Module-local late imports used to avoid Tk/startup cycles are not automatically defective; no proven circular-import crash was found in exercised startup paths.

Suite UI consistency audit — no redesign

Role	Current implementation	Safe centralization target
Toolbar icons and dimensions	ogcore/icons.py already defines canonical names, SIZE=26, cube size=28, label font, padding and spacing; caller helper aliases/tuple-built toolbars also exist	Keep this service. Add an action-to-icon registry and action-level tests. Same function must use exactly the same icon; labels are not a reliable action identity. The literal-call inventory is partial, so it does not prove every dynamic toolbar obeys the rule.

Role	Current implementation	Safe centralization target
Toolbar/menu/check/radio treatment	Shared helpers coexist with plain ttk/tk controls, FlowRows and per-app builders	Define roles for button, toggle, radio, menu and compact action; preserve each window’s workflow/grouping and wrap behavior.
Fonts/backgrounds/borders	ogcore/theme.py is shared by most apps; Petro has a separate _setup_style and constants; local hard-coded variants remain	One token source for base/header/mono fonts, panel/workspace/border colors and spacing, plus explicit variants. Do not change geological plot colors as a cosmetic side effect.
Tabs/panel headers	Shared base notebook padding and frame styles, plus local ribbon/step/card styles	Shared role definitions for tabs, panel titles, group headers and active states. Namespace true exceptions and test open-order effects.
Dropdowns	Icon menubuttons, readonly comboboxes and custom layouts vary in padding/width	Centralize compact/normal role metrics and focus/disabled treatment; preserve selection semantics.
Selection / active state	icons.ON and local selected/active colors; Petro and theme maps differ	Shared selected/active/pressed/disabled tokens with keyboard-focus and high-DPI checks.
Status bars	Local labels/padding and text-variable handling	A small shared style/constructor, preserving each application’s status content and progress behavior.
Scientific canvases	Independent Matplotlib renderers, fonts and units	Keep renderers independent; share only presentation defaults that have an explicit common meaning and regression references.

K01/K02 explain the concrete maintenance concerns. This is an incremental consistency pass, not justification for a total UI overhaul. No confirmed same-function icon mismatch is asserted from the limited literal-call scan; enforce the requested rule through action IDs and include dynamic/tuple-built controls in the later pass.

Prioritized findings

Findings: 2 Critical, 10 High, 9 Medium, 3 Low, 5 Cleanup.

Critical

C01 — Geosteering output filenames collide across different wells

Severity: Critical

Classification: Confirmed defect; reproduced

File(s): geosteering/project.py:537–543; database.py:3250–3331

Function/class: write_results; ProjectDatabase.load_las_file

Current behavior: The results path is made from a sanitized lateral well name, without the database well ID or API. The file is atomically replaced and registered separately for each well.

Why it matters: Well names are not unique, and sanitization also makes distinct names collide. Atomic replacement protects one write from truncation but does not protect the other well’s file.

Realistic failure scenario / evidence: Two project wells named Same Name produced the same Same_Name_geosteering.las. Saving well 2 changed the file used by well 1; both database registrations retained that shared path, with different reported APIs.

Recommended fix: Use stable well identity in generated filenames; verify existing file ownership before replacing it. Preserve or explicitly migrate existing registrations without overwriting the wrong well’s result.

Regression tests: Export two same-name wells and two names that sanitize identically; reopen both through the shared curve reader and verify their independent data and APIs.

Confidence: High

Evidence artifact: core-evidence.json: same_name_geosteering; probes/core_probes.py

C02 — Failed geosteering publication permanently removes previous crossing tops

Severity: Critical

Classification: Confirmed defect; failure injection

File(s): geosteering/project.py:525–562

Function/class: write_results

Current behavior: Writes/replaces the LAS, commits its registration, commits a DELETE of previous generated crossing tops, and then adds replacement tops through separately committing helpers.

Why it matters: The logical publication is not atomic. Failure after deletion leaves the project with missing or partial interpretation and a result LAS that already changed.

Realistic failure scenario / evidence: With an existing generated crossing, an injected failure on the first replacement add left no crossing tops; the new LAS remained on disk. A disk/database failure can reach the same boundary.

Recommended fix: Calculate and validate all crossings first; use db.atomic for the complete database change and a recoverable staged-file publication strategy. Keep the previous file/registration until the new publication succeeds.

Regression tests: Inject failure at LAS registration, first/second top insertion, and final commit. Assert all old tops, curve inventory and file bytes survive, then verify a successful save/reopen.

Confidence: High

Evidence artifact: core-evidence.json: failed_geosteering_write

High

H01 — Magnitude-based fraction detection can override valid units by 100×

Severity: High

Classification: Confirmed numerical hazard; existing policy is intentional

File(s): ogcore/units.py:41–61; ogcore/pay.py:79–90,225–261; ogcore/formulas.py:28–30

Function/class: percent_scale; as_decimal; pay_mask; summarize

Current behavior: For any finite data, the 90th percentile decides percent versus decimal; declared percent and v/v units are ignored. Units are consulted only for empty/all-null data.

Why it matters: Legitimate low porosity or low saturation reported in percent can be treated as decimal, materially changing cutoffs, phi-h and saturation calculations. Conversely, outliers can change an entire decimal curve’s interpretation.

Realistic failure scenario / evidence: PHIE [0.5,1.0] with unit % returned [0.5,1.0], rather than [0.005,0.01]. The code deliberately supports mislabeled Petra exports, and tests encode that policy; changing to units-first globally would also regress supported data.

Recommended fix: Add an explicit persisted fraction-scale interpretation/override with visible provenance and ambiguity checks. Retain a documented auto mode for mislabeled legacy data; do not silently reinterpret existing projects.

Regression tests: Use independent reference cases for low-percent and decimal curves, explicit overrides, outliers, empty curves, and the existing mislabeled-Petra fixture; verify both pay cutoffs and Archie outputs.

Confidence: High

Evidence artifact: core-evidence.json: explicit_percent; tests/test_ogcore.py:401–403 documents current heuristic

H02 — Batch calculations mix raw LAS depths with project well depths

Severity: High

Classification: Confirmed numerical defect; reproduced

File(s): ogcore/batch_zones.py:477–517,643–730; ogcore/batch_curves.py:228–243; database.py:2161–2202

Function/class: _WellCurves.read; calculate_well; curve_values

Current behavior: LAS depth arrays are returned in their file units, while zone boundaries and the directional converter use well/project depth units. Curve resampling compares the raw numbers; the return interface carries curve-value units but not depth units.

Why it matters: Metres and feet can be compared as if identical. This can skip valid intervals or publish incorrect zero pay, misalign curves from different runs, and apply the survey converter to the wrong MD axis.

Realistic failure scenario / evidence: A feet-based well with a 100–110 m LAS and the equivalent 328.084–360.892 ft zone returned NET=0, despite a fully populated physical interval. A required-curve cutoff can instead report inadequate coverage.

Recommended fix: Establish an explicit depth-unit contract at the shared reader boundary; normalize arrays and interval bounds into the same axis before interpolation/calculation. Reject unresolved units rather than guessing. Keep renderers independent.

Regression tests: Use physically identical ft/m LAS runs, feet/metre well records, mixed-run inputs, and a directional survey; compare pay, depth inputs, and saved/reopened calculated curves to independent references.

Confidence: High

Evidence artifact: store-evidence.json: mixed_depth_units; probes/store_probes.py

H03 — Production per-foot metrics are actually per well depth unit

Severity: High

Classification: Confirmed numerical defect; reproduced

File(s): ogcore/production.py:302–351,365–377; ogmap/sources.py:unit

Function/class: gross_spans; summarize; metric_values

Current behavior: Perforation span is read in each well’s depth units and divided directly into volumes. The metrics remain named *_per_ft and one source-unit label is chosen for the result set.

Why it matters: A project containing metre and feet wells produces dimensionally inconsistent scalar values in the same map/statistic. Relabeling the entire variable cannot correct individual wells.

Realistic failure scenario / evidence: For two wells with 1,000 BOE and span 100, one in feet and one in metres, both returned 10 BOE/ft. The metre well should be 3.048 BOE/ft.

Recommended fix: Normalize spans to a documented denominator unit before computing per-foot metrics, or provide explicit unit-specific metrics with per-well conversion. Audit imported historical values before any migration.

Regression tests: Compare identical physical laterals in ft and m and a mixed-unit map, including first-12 metrics and EUR/span; verify source labels and exports.

Confidence: High

Evidence artifact: batch-production-evidence.json: per_foot

H04 — Two batch definitions sharing a source delete each other’s zone results

Severity: High

Classification: Confirmed data-loss defect; end-to-end reproduced

File(s): database.py:2211–2245; ogcore/batch_zones.py:193–194,978–990

Function/class: publish_zone_values; Definition.source_name; batch_zones.run

Current behavior: publish_zone_values treats each call as the complete snapshot for a well/source and deletes every earlier attribute from that source that was not included. Batch definitions allow a user-supplied source and publish one definition at a time.

Why it matters: An interpretation source is being used as a producer/ownership identifier. Different definitions or zones legitimately assigned the same source are not separate snapshots.

Realistic failure scenario / evidence: Job A published Zone A/Gross=10 under TEAM. Job B then published Zone B/Gross=10 under TEAM. Both reported success, but Zone A’s result disappeared.

Recommended fix: Separate producer/job ownership from interpreter/source identity, or explicitly scope replacement to a definition’s owned zone/attributes. Preserve full-snapshot semantics where Petro intentionally publishes its entire result set.

Regression tests: Run two jobs with the same source and different zones/attributes in both orders, rerun one, remove an output, and test skip/rename/overwrite policies and Petro coexistence.

Confidence: High

Evidence artifact: batch-production-evidence.json: Job A, Job B; helper-level confirmation in core-evidence.json

H05 — A newer database schema is opened and stamped as schema 27

Severity: High

Classification: Confirmed forward-compatibility defect; reproduced

File(s): database.py:956–1009

Function/class: ProjectDatabase._open

Current behavior: Reads PRAGMA user_version, executes schema/migration work, and sets any differing version to SCHEMA_VERSION=27. There is no rejection for a larger database schema version.

Why it matters: An older binary can operate on an unsupported newer project and remove its version warning by stamping it backward. Future migrations, triggers or semantics may be incompatible even if table names still match.

Realistic failure scenario / evidence: A synthetic current-schema database marked user_version=99 opened successfully and was changed to 27. This proves the missing guard; it does not prove damage to a particular future schema that does not yet exist.

Recommended fix: Reject newer schema versions before executescript or any writes; provide a clear compatibility message. Test migration entry points and consider a recoverable pre-upgrade backup.

Regression tests: Open a database with a higher version and verify rejection and byte/schema immutability. Preserve successful supported migrations and rollback-on-failure tests.

Confidence: High

Evidence artifact: core-evidence.json: future_schema

H06 — Rotated ZMAP export silently truncates grid extent at 2,000 nodes

Severity: High

Classification: Confirmed numerical/export defect; reproduced

File(s): ogcore/surface.py:125–135,556–582

Function/class: Surface.resampled; write_zmap

Current behavior: For rotated grids, resampling caps nx and ny at MAX_NODES while retaining the original cell size and origin. write_zmap exports that shortened grid without warning.

Why it matters: The cap reduces geographic coverage instead of adjusting spacing to preserve the extent. Exported geological data can be missing over a substantial part of the interpreted area.

Realistic failure scenario / evidence: A rotated 3×3,001 grid spanning X=0–3,000 exported with X=0–1,999 and 2,000 columns. The final third of the extent was absent.

Recommended fix: Preserve the full extent when enforcing node limits by deriving spacing/counts together, or refuse/ask for a larger cell size. State any resampling and preserve coordinate conventions.

Regression tests: Round-trip rotated and negatively oriented grids larger than the cap, compare geographic bounds and a known plane, and test explicit cell sizes and null boundaries.

Confidence: High

Evidence artifact: asset-evidence.json: rotated_zmap_extent

H07 — SEG-Y relinking accepts a geographically different survey

Severity: High

Classification: Confirmed integrity-guard defect; reproduced

File(s): ogcore/segy.py:757–773; seismicworkbench/project.py:80–120

Function/class: SegySurvey.fingerprint; SeismicStore.open_survey; relink_survey

Current behavior: The fingerprint hashes inline/crossline axes, samples, domain, units and line-header bytes, but not map coordinates/placement or the occupied-bin mask.

Why it matters: The advertised geometry guard cannot distinguish surveys with identical bin labels and sampling but different geographic placement. Stored horizons/faults can then be shown against a replacement volume at a different location.

Realistic failure scenario / evidence: Two synthetic surveys whose X/Y coordinates differ by (+10,000,+20,000) had equal fingerprints; relink_survey accepted the replacement.

Recommended fix: Separate bin-topology identity from geographic-placement identity; compare coordinates/affine, units, CRS/placement metadata and occupancy with documented tolerances. Make intentional re-placement an explicit workflow rather than a silent relink.

Regression tests: Relink identical relocated files successfully; reject changed origin/rotation/scalar/CRS and changed missing-bin patterns. Preserve legitimate header-byte correction and explicit bin-listing workflows.

Confidence: High

Evidence artifact: seismic-evidence.json; probes/seismic_probe.py

H08 — Grid file changes survive a failed database save

Severity: High

Classification: Confirmed persistence defect; failure injection

File(s): ogmap/grids.py:139–164,278–309,340–351

Function/class: write_file; save; save_display

Current behavior: Replaces the authoritative .ogwgrid file before updating the database row. File and database persistence have no shared rollback or recovery record; temporary filenames are also fixed per target.

Why it matters: A reported failed save can still alter the grid users reopen. A rename can leave an unregistered file, and concurrent writers can collide on the same .tmp path.

Realistic failure scenario / evidence: A trigger aborting UPDATE map_grids caused save to raise, but the old file was already replaced: reopening returned grid value 9 rather than the previous 1.

Recommended fix: Use staged/versioned assets and a recoverable database/file publication protocol with unique temporary files. Keep the previous asset until metadata publication succeeds and reconcile pending operations on reopen.

Regression tests: Inject file-write, row-update and commit failures for saves, renames and style-only changes; assert old file bytes and row metadata remain consistent. Exercise two concurrent writers.

Confidence: High

Evidence artifact: asset-evidence.json: failed_grid_save

H09 — Correlation and Cross Section retain stale curves and well facts after external edits

Severity: High

Classification: Confirmed cross-module defect; reproduced

File(s): correlationworkbench/app.py:543–584; correlationworkbench/store.py:142–170,196–262; xsection/window.py:115–129; xsection/store.py:112–118,204–218,290–324

Function/class: _focus_back; _focus_in; Store.refresh_picks; Store._well_curves

Current behavior: Focus refresh considers tops/zones/wells, excludes logs, and refreshes only picks. Cached well objects and curve arrays survive. Cross Section has forget(), but this refresh path does not call it.

Why it matters: Edits accepted in Petro, batch calculations or Well Manager can be absent from an open geological view. New curves are not offered; changed curves, names, datums and geometry can remain stale.

Realistic failure scenario / evidence: After another connection replaced GR=[10,20] with [90,95], added NEW, and renamed the well, both stores still returned [10,20], could not resolve NEW, and retained the old name after refresh_picks. A new store read the correct values.

Recommended fix: Invalidate the specific cached facts/curves for their dependency domains and refresh the active view without losing layout or picks. Use revisions/stamps consistently and make refresh behavior

shared policy, not a renderer merge.

Regression tests: Keep both views open while a second connection edits curves, LAS registration, header/datum, survey, core and rasters. Verify updated content, unchanged display layout and no reload for unrelated style edits.

Confidence: High

Evidence artifact: store-evidence.json: after_external_edit_refresh_picks and fresh_read

H10 — Workbook export repeatedly scans every existing cell and scales quadratically

Severity: High

Classification: Confirmed performance defect; measured

File(s): export_manager.py:118–153,156–194

Function/class: _Sheet.append; _append_text_safe; _Workbook; _format_sheet

Current behavior: Each appended row checks Worksheet.max_row, then _append_text_safe checks it again. openpyxl computes max_row by scanning the stored cell keys. The workbook also retains every cell in memory, and formatting slices a whole column before limiting to 4,000 rows.

Why it matters: Large exports have quadratic append cost plus memory proportional to all output cells. Splitting at Excel’s row limit does not bound memory or make the preceding appends efficient.

Realistic failure scenario / evidence: Six-column append probes took 0.184 s/1,000 rows, 0.577 s/2,000 and 2.132 s/4,000. These are isolated construction timings, not a 100,000-well end-to-end benchmark. Large relational exports can freeze the UI or exhaust memory.

Recommended fix: Track row indices explicitly and remove repeated max_row scans; stream/write-only output where compatible with formatting and safe-text handling. Bound width sampling and check cancellation within large sheets. Publish the completed workbook atomically.

Regression tests: Benchmark increasing row counts and assert operation counts/near-linear scaling, memory bounds and cancellation. Verify Excel splitting, API/UID preservation, formula-like text safety, and export/reimport content.

Confidence: High

Evidence artifact: export-evidence.json; probes/export_probe.py

Medium

M01 — Gas unit labels and unsupported unit guesses can produce major conversion errors

Severity: Medium

Classification: Confirmed conversion behavior; intended Mm³ convention is ambiguous

File(s): ogcore/production.py:63,594–610,677–678

Function/class: UNITS; guess_mapping; import_rows

Current behavior: Mm³ maps to 35.3147 Mcf, which is approximately 1,000 m³, not one million m³. The label does not explain a petroleum-thousands convention. Gas (m3) is mapped as gas but its ASCII unit is not recognized; omitted/unknown factors silently default to stored units or 1.

Why it matters: Files using the common million-m³ interpretation can be understated by 1,000×; an unrecognized m3 heading can instead be treated as Mcf. Correctness depends on users catching an undocumented convention in the mapping dialog.

Realistic failure scenario / evidence: Importing 1 with selected Mm³ stored 35.3147 Mcf. One million m³ equals about 35,314.667 Mcf. guess_mapping([Month, Gas (m3)]) returned a gas mapping with no unit mapping. The intended meaning of Mm³ is not independently documented in this archive.

Recommended fix: Use explicit labels such as 10^3 m³ and 10^6 m³, recognize ASCII/superscript aliases, and require acknowledgement of a supplied but unknown unit. Do not automatically rescale historical imports without provenance.

Regression tests: Reference-test every supported gas unit and common ASCII headings; reject unknown units and preserve existing project values. Test explicit user override of guesses.

Confidence: Medium (High for observed behavior; Medium for the intended Mm³ meaning)

Evidence artifact: core-evidence.json: million_cubic_metres; batch-production-evidence.json: ascii_gas_m3_guesses

M02 — Per-session curve and raster caches have no aggregate memory budget

Severity: Medium

Classification: Confirmed unbounded retention policy; large-session risk

File(s): correlationworkbench/store.py:134–137,225–262,298–305; xsection/store.py:104–107,290–324,389–396; seismicworkbench/app.py:1676–1681; ogcore/raster.py:622–635

Function/class: Store._curves; Store._images; curve_values; RasterImage

Current behavior: Stores keep arrays for every visited well and RasterImage instances for every visited file. Each RasterImage may retain its own 128 MiB band cache. Seismic's well-curve dictionary is also not byte-bounded.

Why it matters: Individual caches are useful, but their total grows with the session's visited data rather than the visible set. This is retention/scalability risk, not proof of an unreachable-object leak.

Realistic failure scenario / evidence: Twenty retained raster readers at their configured budget could hold roughly 2.5 GiB of bands. Four thousand wells × 20 curves × 10,000 float64 samples is about 6.4 GB for curve values alone; actual sharing, use and cache fill determine the footprint.

Recommended fix: Introduce a shared byte-budget/LRU policy or release inactive wells/readers while keeping cheap metadata. Measure before choosing limits, and coordinate invalidation with H09.

Regression tests: Visit more wells/rasters than fit in the budget, force GC, assert retained bytes plateau, then revisit evicted items and verify unchanged curves/images and acceptable reread counts.

Confidence: Medium (High on retention policy; Medium on workload-specific impact)

Evidence artifact: Source inspection; estimates are arithmetic bounds, not measured client RSS

M03 — Cold production summaries materialize the entire monthly history as Python dictionaries

Severity: Medium

Classification: Confirmed allocation pattern; scale risk

File(s): ogcore/production.py:437–449

Function/class: _all_summaries

Current behavior: Reads every monthly production row into a by_well dictionary of lists of per-month dictionaries before computing any summaries. The SQL is already ordered by well, but no well is released while scanning.

Why it matters: The cache improves repeat queries, but cold-cache peak memory remains proportional to all raw monthly rows in Python form. Many millions of months can dominate memory before useful results appear.

Realistic failure scenario / evidence: A 100,000-well project with 120 months per well has 12 million rows; dictionary/list/object overhead is additional to SQLite and NumPy data. No claim is made that this exact production volume was benchmarked here.

Recommended fix: Stream ordered rows one well at a time and release them after producing its summary, preserving source precedence and every numerical convention. Retain the existing revision-aware result cache.

Regression tests: Compare streaming results exactly with reference histories, overlapping sources and missing months; benchmark cold/warm time and peak memory on a separate large fixture.

Confidence: Medium (High on allocation pattern; Medium on project impact)

Evidence artifact: ogcore/production.py source; current historical benchmark in docs is not treated as v0.8.37 evidence

M04 — Posted-value selections stay stale in other open Map windows

Severity: Medium

Classification: Confirmed cache/invalidation defect; reproduced

File(s): database.py:94; ogmap/posting.py:98–109; ogmap/window.py:497–549,557–575

Function/class: MAP_DISPLAY_KEYS; posting.save; MapWindow.posting_items; refresh_all

Current behavior: map_data_posting is not watched by the map-display revision trigger. MapWindow caches posting items after first load and does not clear that cache on external refresh. Wells Apply also saves map_well_labels, but that revision does not invalidate _posting_items.

Why it matters: Two open views can show different posting selections/styles even though the project saved correctly. Reopening resolves the discrepancy, which can look like a persistence failure.

Realistic failure scenario / evidence: An initialized window host cached an empty list. A second connection saved an above-well Mowry post; the database loader returned it, posting.save raised no change domain, and the window method still returned [].

Recommended fix: Include posting in display revision dependencies and invalidate/reload the posting-item cache on external display changes. Keep same-window style-only updates cheap.

Regression tests: Open two maps/connections, add/move/hide/edit a post in one, focus the other, and verify its rendered values/positions. Confirm save/reopen and no unnecessary source queries.

Confidence: High

Evidence artifact: posting-evidence.json; probes/posting_probe.py

M05 — Posted values use one Matplotlib Annotation per well rather than the existing batched text path

Severity: Medium

Classification: Measured scaling cost of intended v0.8.37 behavior

File(s): ogmap/view.py:536–566; ogmap/posting.py:27; ogmap/textpaths.py

Function/class: _draw_posts; posting.layout

Current behavior: POST_LIMIT=None intentionally preserves every visible value at its configured point size. _draw_posts creates a separate Annotation for each line; well header labels already use batched text-path collections.

Why it matters: At basin-wide extents, posted-value rendering scales with all selected wells × posting items, adding substantial object creation and draw cost on view changes. This is not a request to restore thinning, zoom-dependent visibility or size changes.

Realistic failure scenario / evidence: An Agg rendering probe of the actual _draw_posts method produced 5,000 annotations and took about 2.24 s to draw, versus 0.43 s for 1,000; construction added 0.24 s and 0.07 s. This is a controlled synthetic renderer probe, not a client-map timing.

Recommended fix: Reuse the existing batched text-path machinery and geometry caches while preserving every label’s value, size, position, order and visibility. Keep any further behavior changes out of this fix.

Regression tests: Compare rendered positions/point sizes and values before/after across zooms and all four sides; test several posting items and 100,000-well memory/draw budgets.

Confidence: High

Evidence artifact: posting-performance.json; probes/posting_perf.py; CHANGELOG.md v0.8.37 states the intended all-text rule

M06 — Deleting a grid or saved list trusts arbitrary stored paths

Severity: Medium

Classification: Confirmed destructive-path boundary; security/distribution risk

File(s): ogmap/grids.py:358–364; ogcore/well_selection.py:364–371; ogcore/project.py:160–184

Function/class: grids.delete; delete_list; Project.load

Current behavior: Grid/list delete operations resolve the database’s file_path and unlink it without checking that the file belongs to the project’s managed asset folder. Manifest folder roles are also accepted without

load-time containment checks.

Why it matters: A stale, externally linked or deliberately altered project can direct a normal Delete operation at a file outside the project. This is local-file deletion through trusted metadata, not an arbitrary-code-execution finding.

Realistic failure scenario / evidence: A synthetic grid row pointing to an external test file caused that external file to be deleted. A malicious project requires the user to open it and invoke the destructive operation; the probe did not touch real user files.

Recommended fix: Separate managed assets from external references, enforce ownership/containment before unlinking, and restrict deletes of external files to an explicit informed action. Validate role paths on manifest load.

Regression tests: Test external absolute paths, traversal, symlinks and linked assets in copied projects; assert unregistering an external reference leaves its file intact.

Confidence: High

Evidence artifact: asset-evidence.json: external_grid_delete

M07 — One malformed grid metadata file aborts grid discovery instead of being isolated

Severity: Medium

Classification: Confirmed robustness defect; reproduced

File(s): ogmap/grids.py:168–205,208–223,371–396

Function/class: read_file; load; register_folder; list_grids

Current behavior: read_file assumes decoded grid.json is a dictionary. Schema/type errors outside its initial try block are not normalized to ValueError; callers generally catch only ValueError.

Why it matters: A single damaged or incompatible file can prevent the whole grid list from opening, hiding valid grids behind an unrelated parser exception.

Realistic failure scenario / evidence: A valid ZIP named bad.ogwgrid containing grid.json=[] made list_grids raise AttributeError: list object has no attribute get.

Recommended fix: Validate metadata types, geometry, shape and payload size at the read boundary, convert expected input failures to a domain exception, and list a bad asset with a useful problem message while continuing discovery.

Regression tests: Include non-object JSON, missing/wrong fields, invalid shapes, inconsistent raw lengths and one bad file beside valid grids. Preserve the damaged file for recovery.

Confidence: High

Evidence artifact: malformed-evidence.json; probes/malformed_probe.py

M08 — Interrupted shapefile overwrite leaves a mixed old/new package

Severity: Medium

Classification: Confirmed import integrity defect; failure injection

File(s): ogmap/project.py:144–167

Function/class: copy_shapefile

Current behavior: When overwrite is selected, each .dbf/.prj/.shp/.shx sidecar is copied directly into the live destination in sequence, without staging or rollback.

Why it matters: A failure can replace attribute/CRS sidecars while retaining old geometry, invalidating an existing layer even though the import failed. The source file remains safe, but the destination package does not.

Realistic failure scenario / evidence: An injected copy failure at .shp left layer.dbf replaced while the old geometry files remained. Different old/new datasets can then have incompatible feature counts or attributes.

Recommended fix: Stage the complete package, validate it, and publish with a recoverable backup/rollback of all affected destination members. Clean up failed new imports without deleting pre-existing files.

Regression tests: Inject failure at each sidecar copy and database registration, and verify every old destination byte is preserved. Include CRS/feature-count changes and renamed imports.

Confidence: High

Evidence artifact: shapefile-evidence.json; probes/shapefile_probe.py

M09 — Unexpected-error dialog falsely guarantees that data did not change

Severity: Medium

Classification: Confirmed misleading error handling

File(s): ogcore/errorlog.py:49–78

Function/class: install.report

Current behavior: Every unexpected callback error displays “Your data were not changed by this error” without inspecting transaction state or the operation that failed.

Why it matters: Several workflows demonstrably commit before failing. The blanket guarantee can cause users to trust a partially changed project and miss the need to inspect or recover it.

Realistic failure scenario / evidence: The geosteering and grid failure probes changed durable state before raising. If surfaced through this handler, the dialog would make the opposite claim.

Recommended fix: Use neutral wording that says the operation failed and identifies the log. Show unchanged/rolled-back guarantees only when the operation can establish them.

Regression tests: Route both a fully rolled-back edit and a partially published file/database failure through the handler; verify truthful messages and preserved diagnostic details.

Confidence: High

Evidence artifact: ogcore/errorlog.py:60–65; core-evidence.json and asset-evidence.json failure cases

Low

L01 — Forgiving LAS parsing drops malformed samples without reporting them

Severity: Low

Classification: Confirmed input-recovery behavior; provenance gap

File(s): ogcore/las.py:189–247

Function/class: parse_data_lines; read_las

Current behavior: Fallback parsing skips lines containing nonnumeric tokens and pads short rows with NaN. No warning/report of skipped rows is returned to callers.

Why it matters: Forgiving input is useful, but users cannot distinguish a complete log from a recovered partial read. A bad row in an important interval may silently disappear.

Realistic failure scenario / evidence: A mostly valid LAS with one damaged numeric line loads its remaining samples successfully. Downstream resampling may then bridge the missing depth unless the gap is represented as NaN.

Recommended fix: Retain tolerant loading but attach parse diagnostics with line/sample counts and make imports surface significant recovery. Do not silently change supported WRAP behavior.

Regression tests: Load malformed, short, wrapped and descending LAS samples and assert both recovered values and explicit diagnostics; verify gaps remain gaps through downstream use.

Confidence: Medium (High on parser behavior; Medium on downstream impact)

Evidence artifact: ogcore/las.py source inspection; not asserted as a client-data failure

L02 — Malformed session-lock metadata can prevent a project from opening

Severity: Low

Classification: Confirmed validation gap

File(s): ogcore/safeio.py:126–148; seismicworkbench/app.py:352–359

Function/class: SessionLock._read; SessionLock.holder

Current behavior: JSON parse errors return None, but a successfully parsed non-dict or nonnumeric pid reaches unchecked .get()/int() operations. The holder call precedes the application's database-open error handler.

Why it matters: A small corrupted advisory lock file can raise instead of yielding a useful stale-lock warning, blocking an otherwise valid project-open attempt.

Realistic failure scenario / evidence: A .seismic.lock containing [1] or a dict with pid="bad" raises an attribute/conversion error when holder() is called. An empty list is falsey and already treated as no lock.

Recommended fix: Validate lock record shape and field types; treat invalid metadata as an explicit recoverable advisory-lock problem, while retaining valid remote-host warnings.

Regression tests: Exercise empty/truncated/non-object records, nonnumeric/negative PID, active local/remote holders, and retry after an invalid lock is safely handled.

Confidence: High

Evidence artifact: ui-diagnostics.json: bad_lock; ogcore/safeio.py source

L03 — The exercised Tk 9 wheel event is interpreted in the wrong direction

Severity: Low

Classification: Confirmed runtime compatibility defect in the synthetic X11 event path

File(s): ogcore/scrollframe.py:140–147,163–190; tests/test_scrolling.py:24–29,93–103

Function/class: _wheel_steps; install_wheel

Current behavior: Button-number fallback treats only num==5 as downward; every other button event means upward. In this Tcl/Tk 9.0.4 environment, generating the test's Button-5 event delivered num=9.

Why it matters: The event is routed to the correct entry and scroll frame but interpreted as upward, so a form at its top does not scroll down. Windows/Tk 8.6 manual input was not tested here.

Realistic failure scenario / evidence: The failing wheel test reproduced in isolation; the diagnostic recorded num=9 (int), the correct TEntry and ScrollFrame, and unchanged yview at 0.

Recommended fix: Normalize wheel direction for the explicitly bound sequences and supported Tk versions, or document/pin the supported toolkit until compatibility is verified. Do not blanket-map unknown buttons without checking real input.

Regression tests: Test direction normalization and manual/synthetic wheel paths on supported Windows Tk and X11 Tk 8.6/9, including horizontal scroll and list widgets that consume their own wheel.

Confidence: High (High for this exercised path; Medium for broader Tk 9 input impact)

Evidence artifact: ui-diagnostics.json

Cleanup

K01 — UI style roles are partly centralized but redefined with conflicting global names

Severity: Cleanup

Classification: Architectural concern; cosmetic consistency, not a correctness defect

File(s): ogcore/theme.py:1–76; ogcore/icons.py:44–58,145–175; petroworkbench/app.py:200–252; correlationworkbench/app.py:88–99; xsection/window.py:264–271

Function/class: theme.apply; icons.icon_style; PetroWorkbench._setup_style; toolbar builders

Current behavior: Theme and icon geometry already have shared services. Petro separately defines base/button/menu styling; Correlation and Cross Section both configure global Tool.TButton and Ribbon styles with different padding/width/font values.

Why it matters: ttk style definitions apply to the interpreter, so opening another module can alter controls using the same names. Repeated role definitions make a later consistency pass harder to review.

Realistic failure scenario / evidence: A Geo-hosted Cross Section can redefine a generic style used by other windows in that interpreter; independent Petro windows also differ in base font/padding even for the same role.

Recommended fix: Extend the existing theme/icon services with explicit control roles and shared tokens; namespace truly application-specific variants. Define the same action’s icon once, then migrate callers incrementally.

Regression tests: Instantiate representative toolbars in both open orders and compare role metrics, icon IDs, font, padding, selected/disabled states and narrow-window wrapping. Preserve workflow-specific layouts.

Confidence: High

Evidence artifact: Source inspection; icon-inventory.json contains literal icon calls, not a complete dynamic-action registry

K02 — Core field metadata is coupled to Geo UI classes

Severity: Cleanup

Classification: Contained architectural concern

File(s): correlationworkbench/store.py:30–48; geoworkbench/well_tab.py:WellTab.FIELDS

Function/class: well_label_fields

Current behavior: Correlation imports WellTab solely to get field descriptors, catching any exception and falling back to a smaller hard-coded set of fields.

Why it matters: A UI import failure changes the data fields offered by another application and creates duplicated metadata. This is a concrete maintenance hazard, but no present geological-value defect was reproduced from it.

Realistic failure scenario / evidence: A future Geo UI dependency/import change can silently shrink Correlation’s label choices while its underlying database remains valid.

Recommended fix: Move the stable well-field descriptors into a small data-only registry consumed by the existing forms and stores. Keep their view models and widgets separate.

Regression tests: Assert identical field IDs/order and existing saved-label resolution before/after, including standalone Correlation startup without constructing a Geo window.

Confidence: High

Evidence artifact: correlationworkbench/store.py source; no broad model unification recommended

K03 — Several shipped helpers have no internal caller and can be retired after API review

Severity: Cleanup

Classification: Confirmed absent internal references; external use not knowable

File(s): geoworkbench/common.py:56–65; seismicworkbench/windows.py:293–296; ogcore/batch_zones.py:589–592

Function/class: well_type_label; well_type_from_label; make_toolbar; zone_interval

Current behavior: These functions appear only at their definitions in the shipped Python source/tests; the two well-type helpers are also mentioned as candidates in the old audit document. zone_interval simply wraps zone_boundaries.

Why it matters: They add unused compatibility surface and can mislead future work toward a path the application does not use. Static absence alone is not proof that an external integration never imported them.

Realistic failure scenario / evidence: A developer updates one of these helpers expecting current UI/calculation behavior to change, but no shipped workflow calls it.

Recommended fix: Confirm they are not supported external API, then remove or clearly deprecate them in a small isolated change. Do not delete migration paths or renderer code merely because a static tool flags it.

Regression tests: Run import/startup smoke tests and affected Geo/Seismic/batch tests; use a repository-wide reference check including dispatch strings and documentation.

Confidence: High (High for no shipped callers; Medium for deletion safety)

Evidence artifact: dead-code-candidates.json is a candidate list, not a deletion list; selected references were manually checked

K04 — Historical audit comments and an unused posting limit contradict the current baseline

Severity: Cleanup

Classification: Confirmed obsolete documentation/constant

File(s): ogmap/view.py:507; docs/REFACTORING_AUDIT.md; docs/MAP_PERFORMANCE.md; CHANGELOG.md:7–11

Function/class: MapView.LABEL_LIMIT; historical audit documents

Current behavior: LABEL_LIMIT=600 still describes bubble-value posting limits although current value posting uses POST_LIMIT=None and bubble-value posting has changed. Historical documents describe older caching/LOD behavior and benchmark versions.

Why it matters: The old measurements are useful history but are not evidence of current v0.8.37 behavior. Stale comments can lead a later fix to restore intentionally removed limits.

Realistic failure scenario / evidence: A future performance pass uses the historical label-thinning recommendation and unintentionally violates the v0.8.37 rule that text stays at its given point size and remains visible.

Recommended fix: Remove the genuinely unused constant/comment after reference verification; label historical measurements explicitly and add a current benchmark record. Preserve intentional compatibility readers.

Regression tests: Verify no live reference to LABEL_LIMIT; retain map-text point-size/visibility tests and compare current benchmark workloads without changing their behavior.

Confidence: High

Evidence artifact: Repository-wide reference search; CHANGELOG.md v0.8.37 is the current behavior baseline

K05 — Tests assume writable user caches, elapsed event time and specific toolkit representations

Severity: Cleanup

Classification: Confirmed test-harness weaknesses; not blanket product failures

File(s): tests/conftest.py; tests/test_ui_lag.py:150–163; tests/test_startup.py:258; tests/test_scrolling_map.py:check_small; tests/test_scrolling_seis.py:shrink

Function/class: test fixtures; GUI progress and geometry assertions

Current behavior: Raster tests do not all isolate OGW_RASTER_CACHE. One spreadsheet test expects a 1 ms timer to finish after 20 update calls. Some dialog tests require scrolling even when content fits. A Treeview test compares whole item dictionaries including the empty-text representation.

Why it matters: These assumptions produce failures unrelated to data correctness and can mask actual regressions in noisy results. Conversely, image invariance tests are valuable and their residual pixel differences still need review rather than arbitrary tolerance changes.

Realistic failure scenario / evidence: Three raster failures passed with a writable cache; the spreadsheet failure passed on targeted rerun. GridDialog’s 193 px body fit its 209 px canvas with all buttons reachable, yet the test required scrolling. Tk 9 returned empty text as () versus an empty string.

Recommended fix: Make cache fixtures hermetic, wait for conditions with bounded deadlines, assert accessibility when content fits, and normalize documented toolkit representations. Keep backend-specific visual tests and Windows CI coverage.

Regression tests: Add fixture isolation checks and deterministic event-loop tests. Keep exact numerical/persistence assertions; do not disable failing tests or merely weaken all visual comparisons.

Confidence: High

Evidence artifact: test-results.xml; retest-results.xml; ui-diagnostics.json; dialog-diagnostics.json

Test coverage recommendations

The existing suite is substantive: it contains external-connection and persistence tests, independent analytic grid cases, multiple SEG-Y formats, failure-injected atomic pay edits and real public LAS fixtures. The

highest-value additions are not more assertions that an internal helper was called:

1. Two-process/connection change tests for cached curves, new/deleted registrations, headers, surveys, core/raster availability and posting settings. Assert the rendered/queried new content and preservation of layouts, not only that refresh was invoked.

2. Mixed-unit physical-reference fixtures: the same well/log/zone in metres and feet; declared low-percent curves; per-foot production; source-unit labels in mixed projects. Keep current mislabeled-Petra cases as compatibility tests, with explicit interpretation choices.

3. Failure matrices for combined asset/database publication: replacement tops, grid rename/style/save, and shapefile package overwrite. Snapshot file bytes and authoritative rows before each injected failure.

4. Duplicate identity versus duplicate display-name tests: same-name wells, sanitized name collisions, stable UID/API matching, multiple batch definitions assigned the same interpreter.

5. Geometry integrity tests for SEG-Y relinking and rotated-grid export that compare coordinates/extents and known geological planes, not just array shape or axis labels.

6. A separate scale suite for 4,000 active Petro/Geo wells and 100,000+ available Map/Seismic wells, combined with 200 layers, multiple grids, large LAS/raster sets and cold production summaries. Measure cold/warm latency, peak and retained bytes, callbacks/artists after repeated close, and I/O/SQL counts. Do not turn machine-specific milliseconds into brittle ordinary unit assertions.

7. Restore a distributable, permission-cleared set of real raster images for the seven skipped real-image tests, or keep a private documented validation job. Do not claim the skipped interactions were exercised by generic image fixtures.

8. Keep a Windows/Tk build job for exact platform behavior; make Linux toolkit checks explicit. K05 describes cache and event-loop fixes. The residual Correlation pixel mismatch needs visual/reference review before adjusting tolerance.

Existing tests such as map-cache call counts and artist counts are useful for performance contracts, but they are insufficient alone: a source dictionary can be correct while the UI saves another source, and an artist can exist while its offsets, extent, values or visibility are wrong. Pair those contracts with content and persistence assertions. Historical tests that intentionally encode data-driven fraction scaling should be retained as auto-mode behavior, not taken as evidence that every declared unit is interpreted correctly.

Security, distribution and resource notes

- ogcore/formulas.py checks expression AST nodes, restricts NP attribute names, rejects strings/other attribute access and removes builtins before eval. No arbitrary Python execution through a standard saved equation was demonstrated. NumPy project/cache loads inspected use allow_pickle=False. Keep those boundaries; no licensing system is proposed.

- Subprocess launch paths inspected use argument lists for bundled applications/system file opening rather than shell=True. This is not a claim that opening arbitrary external files is harmless; no shell-injection exploit was found in those paths.

- Grid/list deletion and manifest role paths need ownership validation (M06). Grid/archive schema and payload bounds need validation (M07); resource exhaustion is a separate concern from code execution.

- Runtime requirements are lower-bound ranges; the Windows build upgrades dependencies and merges several PyInstaller outputs. This audit did not establish a broken package, but a versioned lock/build manifest, third-party license inventory and the existing executable self-tests should be part of release qualification. No hard-coded developer-home dependency or embedded credential was identified in the inspected launch/configuration paths. This is not a secret-scanning attestation. The LAS fixtures are documented as public data.

- Raster LIC files are control files, not software licenses. No conclusion about license enforcement or bypass resistance is warranted from them; no new licensing design is requested.

- Existing safeio writes use unique same-directory temporary files, fsync and replace. Several higher-level asset workflows do not extend that guarantee across all files and database rows. Fix those logical operations rather than replacing the safe helper.

- Many event connections are established once at canvas creation, and close paths cancel jobs/unsubscribe/remove artists. No general orphan-artist or duplicate-signal leak was established. M02 is retained-data growth, not proof of a Python leak. A long-session resource budget test is still needed.

- Session locks are advisory holder checks followed by writes, and some UI paths deliberately allow a second session with a warning. That is not a lost-update prevention mechanism. Before endorsing concurrent interpretation editing, test revision/conflict detection at save; do not silently change the existing workflow based only on this audit.

Evidence and reproducibility

The evidence bundle contains findings.json, core/store/asset/seismic/batch/production/posting/shapefile/malformed-grid evidence, two performance measurements, UI diagnostics, the original suite and targeted-rerun XML/logs, source integrity verification and the external probe scripts. Probes create only temporary synthetic projects. Some use unittest.mock to force a failure at a particular operation; the application source remains unchanged.

External review helper scripts are not proposed application tests or implementation changes. The venv and display server are temporary audit infrastructure. To repeat the test workflow on this host: use the runtime requirements and pytest in an external environment, a usable Tk display, PYTHONDONTWRITEBYTECODE=1, an external MPLCONFIGDIR, OGW_RASTER_CACHE and color-bar library, and pytest -p no:cacheprovider. The exact commands and dependency versions are in environment.json. Do not open client databases with ordinary ProjectDatabase for a forensic read-only check: its normal constructor performs migrations and LIC maintenance; use an isolated copy or its read-only reader where appropriate.

Top 10 Issues to Fix Before v1.0

1. **Protect each well’s generated geosteering file identity** — C01. Stop cross-well overwrites before any output-path migration.

2. **Make geosteering results publication recoverable and atomic** — C02. Preserve old files/registrations/tops when a save fails.

3. **Separate batch-result ownership from interpreter/source labels** — H04. Stop unrelated results disappearing after successful runs.

4. **Establish explicit numerical-unit contracts and interpretation overrides** — H01–H03, M01. Fix physical-axis and denominator errors without blindly reinterpreting saved client data.

5. **Refresh Correlation/Cross Section facts and curves on relevant external changes** — H09; include M04 posting-cache invalidation in the same dependency review.

6. **Make grid file/database publication recoverable** — H08. Include rename/style operations, unique temporaries and managed-file ownership checks.

7. **Preserve full geographic extent in rotated ZMAP exports** — H06. Confirm a plane and bounds, not just node counts.

8. **Strengthen SEG-Y relink identity checks** — H07. Distinguish unchanged topology, changed geographic placement and intentional re-placement.

9. **Refuse unsupported newer database versions before writing** — H05. This is a small compatibility boundary with high protection value.

10. **Remove quadratic workbook append work and bound export memory** — H10. Then qualify 100,000-well export/cancellation and improve aggregate cache budgets (M02/M03).

Safe to Leave Alone

- Independent Map, Petro, Correlation, Cross Section and Geosteering renderers and view models. Their different responsibilities are legitimate; shared read contracts/invalidation can be improved without merging them.

- database.py and large UI modules merely because they are large. Extract only a concrete contract needed for a verified defect.

- The established subsea-elevation sign convention, minimum-curvature routine, clipped sample-support/weighted-pay definitions, source-selection behavior and petrophysical formula library. The audit does not establish a general algorithm defect in these; correct the unit boundary findings rather than rewriting equations or geological workflows.

- Shared API/UID identity helpers, alias resolution, accepted-calculated-curve precedence, pay/selection services, project folder roles, rotating logs, background stale-result checks, compact Map store and viewport spatial services. Retain their tests and close the specific gaps identified.

- SEG-Y chunked/memory-mapped reads, bounded section cache and resource close paths. The relink fingerprint issue does not imply that cubes should be loaded into memory or the reader replaced.

- Useful compatibility readers/migrations: legacy project layouts, Bubble/coverage definitions, old isopach names and Petro sidecar/alias migration. Current Bubble audit-fix tests cover earlier reported issues; old findings were not automatically carried into this release.

- v0.8.37’s intentional fixed-point-size/all-visible-text policy. Improve batching and resource use without reintroducing thinning, zoom hiding or automatic resizing.

- Existing theme/icons/FlowRows/scroll-frame infrastructure. Extend consistent roles and an action-to-icon mapping; do not start a replacement UI framework.

“Safe to leave alone” means no current evidence justifies modification in this audit, not a guarantee that the area is defect-free or fully validated on every client dataset.

Recommended Fix Order

1. **Freeze the baseline and preserve representative project copies.** Capture current behavior and add failure/reference tests for C01/C02/H04 before changes. Keep every production fix small and independently reviewable.

2. **Stop destructive writes first.** Fix generated-file identity, geosteering publication and batch ownership. Add the newer-schema guard. Audit existing same-path registrations and affected source results diagnostically; do not mass-rewrite client data automatically.

3. **Fix unit contracts with explicit provenance and compatibility choices.** Introduce persisted interpretations/conversions, then update batch/pay/production consumers one at a time against independent references. Recalculate historical client results only through an approved, reviewable workflow.

4. **Fix cross-module invalidation.** Refresh the appropriate facts/curves/posting items on revision changes; preserve layouts and avoid full reloads for unrelated changes. Coordinate cache eviction and invalidation tests.

5. **Make assets and exports reliable.** Grid file/row publication, shapefile overwrite rollback, ownership checks, malformed-asset isolation, full ZMAP extents and SEG-Y relink checks. Test save/reopen and interrupted operations together.

6. **Address measured scale costs.** Fix workbook row tracking first; then streaming export, aggregate caches, cold production memory and posted-value batching. Preserve v0.8.37 appearance and numerical outputs.

7. **Stabilize validation and release qualification.** Hermetic caches, bounded event waits, real-raster fixtures, Windows/Tk reference runs, combined scale workloads and reproducible dependency/build records. Review the unresolved visual invariant before release.

8. **Do the Suite UI Consistency pass last.** Extend the existing style/action services, enforce the same-function/same-icon rule, and migrate a small set of roles per change. Remove only the verified unused helpers after API review. No application code was changed during this audit.